

CYA N

Choose Your AI Noob

Documentazione Tecnica Architettuale

Versione 7.5.0 (Sviluppo)

Progetto Multi-Agent

Agosto 2026

Indice

1	Panoramica Architetture e Vincoli di Sistema	3
1.1	Definizione del Sistema	3
1.2	Requisiti di Resilienza e Operatività Offline	3
1.3	Vincoli Hardware e Scelte Progettuali	4
2	Topologia del Sistema e Grafi di Instradamento	5
2.1	Mappatura delle Relazioni Gerarchiche	5
2.2	Struttura del Filesystem e Archi di Dipendenza	5
3	Il Classificatore Neurale (nn_classifier.py)	8
3.1	Motivazione Architetture e Confronto con il Router LLM	8
3.2	Architettura MultiTaskMLP	8
3.3	Formato della Stringa di Input e Ruolo di history_utils.py	9
3.4	Meccanismo di Decisione a Due Stadi (_derive_class_id)	9
3.5	Scaricamento e Gestione della Memoria	10
4	Pipeline di Addestramento del Classificatore	11
4.1	Panoramica della Cascata di Preprocessing	11
4.2	Generazione del Dataset (build_dataset_v2.py)	11
4.3	Pre-calcolo degli Embedding (precompute_embeddings.py)	12
4.4	Addestramento (train_nn.py)	12
4.5	Valutazione Qualitativa (step4_evaluation.py)	13
5	Esecuzione della Pipeline Sequenziale in Ambienti a Memoria Vincolata	14
5.1	Architettura Draft & Merge: Esecuzione Asincrona	14
5.2	Sincronizzazione Attiva e Gestione dello Scaricamento (RAM)	14
5.3	Passaggio di Consegne (Handoff) e Salvaguardia del Contesto	15
6	Chat History e Persistenza Conversazionale	16
6.1	Architettura della Memoria di Sessione	16
6.2	Meccanismo di Sliding Window	16
6.3	Integrazione e Iniezione del Contesto	17
6.4	Integrità dello Stato e Controllo Utente	17

7	Gestione del Cambio di Dominio e Isolamento del Contesto	18
7.1	Principio di Routing Puro	18
7.2	Isolamento della History su Domain Switch	18
7.3	Aggiornamento dello Stato di Sessione	19
8	Critic Pass, Analisi Strutturata e Sanificazione dell'Output	20
8.1	Revisione Critica: Validazione Antagonistica (Critic Pass)	20
8.2	Parsing Strutturato e Gestione dei Tag di Ragionamento	20
8.3	Intercettazione delle Eccezioni a Basso Livello	21
9	Topologia della Flotta LLM e Parametri di Inferenza	22
9.1	Provisioning e Gerarchia dei Modelli	22
9.2	Calibrazione delle Temperature (Determinismo vs Entropia)	23
9.3	Sistema di Prompt Tiering	23
10	Istruzioni Operative: Messa in Produzione	25
10.1	Inizializzazione del Motore di Inferenza (Ollama)	25
10.2	Provisioning della Flotta: Scaricamento dei Modelli	25
10.3	Addestramento del Classificatore Neurale	25
10.4	Configurazione dell'Ambiente e Avvio	26
11	Glossario Architetturale	27

Capitolo 1

Panoramica Architetture e Vincoli di Sistema

1.1 Definizione del Sistema

CYA N (Choose Your AI Noob) è un orchestratore intelligente per Large Language Models (LLM), progettato specificatamente per operare in modo esclusivo all'interno di ambienti di calcolo locali. Alla ricezione di un input testuale, il sistema non effettua un semplice inoltramento (passthrough) della richiesta verso un modello linguistico generico, ma ne analizza la semantica per determinare un percorso di instradamento ottimale.

La classificazione della query è affidata al modulo `nn_classifier.py`, che implementa un classificatore neurale leggero (`MultiTaskMLP`) addestrato su embedding frozen del modello `paraphrase-multilingual-MiniLM-L12-v2`. Il classificatore restituisce un `class_id` strutturato, comprensivo dei punteggi di probabilità per i quattro domini base (coding, math, rights, general), un indice di difficoltà (1-3) e un flag `is_followup` per la gestione del contesto multi-turno. Il `class_id` è l'unica fonte di verità per il routing: nessun modulo esterno lo altera a posteriori.

Il nucleo avanzato del progetto risiede nella sua capacità di elaborare interrogazioni interdisciplinari: l'architettura supporta flussi di esecuzione multi-dominio tramite una "Pipeline Sequenziale", permettendo a più agenti specializzati di collaborare alla risoluzione di una singola richiesta complessa. Qualora il classificatore non sia disponibile (file dei pesi assente), il turno corrente viene scartato con un messaggio di errore esplicito: non esiste più un arco di fallback a parole chiave.

1.2 Requisiti di Resilienza e Operatività Offline

Un requisito non negoziabile di CYA N è la sua totale indipendenza dalle infrastrutture cloud o di rete esterne. Il software instaura processi operativi che garantiscono il 100% delle funzionalità in modalità offline. Ogni scambio di dati — dalla classificazione semantica tramite il `MultiTaskMLP` locale, fino alla generazione dell'output finale da parte dei modelli generativi — viene processato interamente all'interno dell'ambiente hardware locale. Questo approccio architetturale riduce a zero la la-

tenza di rete e offre una garanzia assoluta in termini di privacy e sicurezza dei dati processati.

1.3 Vincoli Hardware e Scelte Progettuali

Lo sviluppo di CYA N ha richiesto l'adattamento dell'infrastruttura a un vincolo hardware rigoroso: garantire un'esecuzione stabile su macchine di livello consumer dotate di soli 8 GB di memoria RAM. Per soddisfare questo requisito, l'architettura adotta specifiche soluzioni ingegneristiche di ottimizzazione:

- **Classificatore Neurale a Bassa Impronta:** La classificazione semantica è affidata al `MultiTaskMLP` (145K parametri), che richiede meno di 100 MB di RAM e conclude l'inferenza in meno di 20 ms. Il modulo viene esplicitamente scaricato (`unload_router()`) dopo la classificazione, liberando l'encoder `MiniLM` (470 MB) prima dell'attivazione dei modelli generativi.
- **Agenti Stateful (Pre-istanziamento):** Al fine di ottimizzare i tempi di risposta, le istanze dei modelli non vengono distrutte e ricreate a ogni ciclo di esecuzione. Esse mantengono uno stato temporaneo in memoria (keep-alive), riducendo le operazioni di lettura (I/O) sul disco e predisponendo l'architettura per la gestione dello storico conversazionale (Chat History).
- **Monitoraggio Dinamico e Sincronizzazione Attiva:** Il sistema esegue controlli preliminari prima di allocare memoria. Tramite la libreria `psutil`, viene interrogato lo stato dell'hardware; in caso di memoria insufficiente, il sistema effettua un declassamento (downgrade) preventivo verso un modello generativo più leggero. Nelle esecuzioni multi-agente, un meccanismo di polling attivo sospende momentaneamente il processo, attendendo che la RAM del primo agente venga liberata prima di inizializzare il secondo, azzerando le probabilità di interruzioni per Out of Memory (OOM).

Capitolo 2

Topologia del Sistema e Grafi di Instradamento

2.1 Mappatura delle Relazioni Gerarchiche

Il flusso di controllo logico di CYA N segue la topologia di un grafo orientato. Il classificatore neurale agisce come nodo di smistamento principale, producendo un `class_id` che identifica univocamente il percorso di esecuzione: un singolo nodo specializzato (esecuzione mono-dominio, `class_id` 0–3) oppure una coppia di nodi in sequenza (Pipeline, `class_id` 4–6). L’ordine esecutivo degli agenti in una pipeline è codificato direttamente nei `class_id`: $4 \Rightarrow \text{math} \rightarrow \text{coding}$, $5 \Rightarrow \text{rights} \rightarrow \text{coding}$, $6 \Rightarrow \text{rights} \rightarrow \text{math}$. Nessuna matrice di configurazione esterna interviene a modificare questa corrispondenza.

Il dominio `general` è strutturalmente escluso dalle pipeline: la funzione `_derive_class_id()` valuta come candidati ai `class_id` pipeline solo i domini tecnici (`coding`, `math`, `rights`, indici 0–2), impedendo per costruzione che il dominio generalista generi instradamenti ibridi.

2.2 Struttura del Filesystem e Archi di Dipendenza

Per assicurare scalabilità e manutenibilità, il progetto implementa una rigorosa separazione delle responsabilità (Separation of Concerns). Ogni modulo sorgente espone interfacce ben definite e instaura dipendenze unidirezionali:

- **main.py (Modulo di Esecuzione e Coordinamento):** Rappresenta l’entry point dell’applicazione (CLI). Gestisce il loop di interazione con l’utente e coordina la logica di instradamento richiamando il classificatore neurale. È responsabile dell’orchestrazione della Pipeline Sequenziale, dell’isolamento della chat history sui domain switch, del troncamento per-messaggio della history (`_truncate_for_history()`, si veda il Capitolo sulla Chat History) e della gestione sicura delle eccezioni hardware (`ResourceExhaustedError`). Il `class_id` prodotto dal classificatore è l’unica variabile che determina il percorso esecutivo: `domain_scores`, `difficulty` e `is_followup` sono utilizzati esclusivamente a scopo diagnostico.

- **config.py (Configurazione Centralizzata):** Agisce come singola fonte di verità statica per il sistema. Definisce le costanti hardware (soglie di RAM), la configurazione dei modelli generativi (LLM primari, di fallback e temperature), il livello di arricchimento del system prompt per dominio (`prompt_tier`, si veda il Capitolo sulla Topologia della Flotta LLM), i parametri della pipeline e le soglie del classificatore neurale (`NEURAL_CLASSIFIER_SETTINGS`).
- **nn_classifier.py (Classificatore Neurale):** Modulo di classificazione basato su MultiTaskMLP. Implementa la funzione pubblica `predict()`, che carica l'encoder frozen, calcola l'embedding della stringa di input costruita da `history_utils.build_input_str()` e inferisce `class_id`, confidenza, `domain_scores`, `difficulty` e `is_followup`. Definisce le costanti pubbliche `DOMAIN_NAMES` e `PIPELINE_CLASSES`. Espone inoltre `unload_router()` per il rilascio esplicito della RAM.
- **history_utils.py (Fonte Canonica della Stringa di Input):** Modulo condiviso tra la fase di training (`precompute_embeddings.py`) e quella di inferenza (`nn_classifier.py`). Espone `build_input_str(query, history_queries)` e la costante `HISTORY_MAX_TURNS` (default 2). Garantisce che la stringa passata all'encoder in produzione sia identica a quella usata durante l'addestramento, eliminando il rischio di deriva semantica silenziosa.
- **ai_engine.py (Motore di Inferenza e Contesto):** Gestisce il ciclo di vita dei modelli linguistici locali. Definisce la classe astratta `BaseAI` e le sue specializzazioni. Implementa la routine principale di generazione testuale, che gestisce lo streaming in tempo reale da Ollama, il parsing dinamico dei tag strutturati (es. `<think></think>`) e il monitoraggio preventivo della memoria tramite soglie di tolleranza. In fase di inizializzazione, ciascuna istanza legge inoltre staticamente il proprio `prompt_tier` dal dominio di appartenenza, usato per modulare l'arricchimento del system prompt (Sistema di Prompt Tiering, Capitolo sulla Topologia della Flotta LLM).
- **prompts_templates.py (Gestore dei Modelli Istruttivi):** Centralizza la definizione del comportamento degli agenti (System Prompts). Mantiene i template dinamici (Few-Shot) richiesti per la costruzione del contesto operativo e per l'esecuzione dei compiti specifici della pipeline, come il passaggio di consegne (Handoff) e la revisione critica (Critic Pass). Espone `get_prompts(category, tier)`, che applica additivamente i blocchi di `TIER_INJECTIONS` quando il dominio è configurato con tier `extended`.
- **helper.py (Libreria di Utilità e Sanificazione):** Modulo dedicato al raffinamento dell'output testuale. Utilizza routine basate su regex per sopprimere tag di sistema e caratteri non supportati. Implementa un dizionario di conversione per trasformare la sintassi matematica strutturata (LaTeX) in caratteri Unicode leggibili, gestendo l'interfaccia utente asincrona tramite un indicatore di caricamento (Spinner) in thread separati.
- **db_query.py (Sorgente di Dati Linguistici):** Contiene le strutture dati predefinite utilizzate per il training del classificatore neurale. Definisce le li-

ste di frasi specifiche per ogni dominio (`INTENT_SENTENCES`) e il dizionario `BRIDGE_SENTENCES` per le query ibride inter-dominio.

- **build_dataset_v2.py (Generatore Unificato del Dataset):** Script di pre-processing che aggrega i dati da `db_query.py`, i record manuali (`MANUAL_RECORDS`) e le augmentazioni lessicali, producendo `dataset_v2.jsonl`. Applica lo split stratificato 70/15/15 *prima* dell'augmentation per prevenire il data leakage.
- **precompute_embeddings.py (Pre-calcolo degli Embedding):** Legge `dataset_v2.jsonl`, codifica ogni stringa di input tramite `paraphrase-multilingual-MiniLM-L12-v2` usando `history_utils.build_input_str()`, e salva il tensore degli embedding e i tensori delle label in `classifier/embeddings_v2.pkl`.
- **train_nn.py (Training del Classificatore Neurale):** Addestra il `MultiTaskMLP` sugli embedding pre-calcolati. Produce il checkpoint `classifier/nn_weights.pt` selezionando il modello con il miglior F1-macro sul validation set (early stopping, patience 25).
- **step4_evaluation.py (Valutazione Qualitativa):** Suite di 59 smoke test strutturati in 6 categorie (mono-domain, pipeline, false pipeline, follow-up, domain switch, edge case). Produce un report diagnostico con accuracy per categoria e analisi delle soglie pipeline.

Capitolo 3

Il Classificatore Neurale (nn_classifier.py)

3.1 Motivazione Architetture e Confronto con il Router LLM

Le versioni precedenti di CYA N affidavano la classificazione semantica a un micro-LLM (`qwen2.5:1.5b`) invocato tramite Ollama. Questo approccio, pur semanticamente robusto, imponeva un overhead significativo: circa 300–800 ms di latenza per classificazione e un'impronta in memoria di 1.5 GB, costringendo il sistema a scaricare il router prima di poter caricare qualsiasi agente generativo.

La versione 7.2.0 sostituisce il router LLM con un classificatore neurale leggero (`MultiTaskMLP`), addestrato su embedding frozen di `paraphrase-multilingual-MiniLM-L12-v2`. Il risultato è un'inferenza in meno di 20 ms con un'impronta inferiore a 100 MB, mantenendo un'interfaccia pubblica (`predict()`, `DOMAIN_NAMES`, `PIPELINE_CLASSES`) identica a quella del router LLM precedente, così da non richiedere alcuna modifica a `main.py`.

3.2 Architettura MultiTaskMLP

Il classificatore è un Multi-Layer Perceptron multi-task composto da un backbone condiviso e tre teste specializzate:

- **Backbone condiviso:**

- Strato lineare $384 \rightarrow 256$, LayerNorm, ReLU, Dropout(0.3)
- Strato lineare $256 \rightarrow 128$, LayerNorm, ReLU, Dropout(0.2)

L'input è un vettore L2-normalizzato a 384 dimensioni prodotto dall'encoder MiniLM frozen. La Layer Normalization stabilizza l'addestramento in presenza di embedding pre-addestrati a distribuzione fissa; i Dropout prevengono l'overfitting su un dataset di dimensioni moderate.

- **Domain Head** ($128 \rightarrow 64 \rightarrow 4$, multi-label): Produce quattro logit indipendenti per i domini coding, math, rights, general. Usata con `BCEWithLogitsLoss`; la Sigmoid viene applicata solo a inferenza per derivare le probabilità `domain_probs`.
- **Difficulty Head** ($128 \rightarrow 32 \rightarrow 3$, classificazione a 3 classi): Stima il livello di difficoltà della query (1–3). Usata con `CrossEntropyLoss`; la Softmax viene applicata solo a inferenza.
- **Is-Followup Head** ($128 \rightarrow 1$, classificazione binaria): Determina se la query è un follow-up diretto della risposta precedente. Usata con `BCEWithLogitsLoss`; la Sigmoid viene applicata solo a inferenza.

Il parametro totale del modello è circa 145.000. Tutti gli head restituiscono logit grezzi durante il forward pass; le funzioni di attivazione sono applicate esclusivamente in `nn_classifier.py` al momento dell'inferenza.

3.3 Formato della Stringa di Input e Ruolo di `history_utils.py`

L'encoder MiniLM riceve in ingresso una singola stringa, costruita dalla funzione `build_input_str()` definita in `history_utils.py` — la *stessa* funzione utilizzata in fase di addestramento da `precompute_embeddings.py`. Questa identità è il vincolo architetturale più critico del sistema: qualsiasi divergenza tra la stringa di training e quella di inferenza altera l'embedding silenziosamente, degradando l'accuratezza in produzione senza sollevare errori.

Il formato adottato è il seguente:

- Se la chat history è vuota: la stringa coincide con la query corrente.
- Se sono presenti turni precedenti: `[HISTORY] q_n-2 | q_n-1 [QUERY] query_corrente`

La profondità massima della history considerata per il classificatore è `HISTORY_MAX_TURNS = 2`, definita come costante unica in `history_utils.py`.

3.4 Meccanismo di Decisione a Due Stadi (`_derive_class_id`)

La funzione `_derive_class_id()` trasforma il vettore di probabilità `domain_probs` in un `class_id`. La logica opera in due stadi con soglie asimmetriche, entrambe lette a runtime da `config.NEURAL_CLASSIFIER_SETTINGS`, dove sono staticamente definite come `threshold_mono = 0.50` e `threshold_pipeline = 0.75`:

- **Stadio 1 — Candidatura** (`DOMAIN_THRESHOLD = 0.50`, permissivo): Vengono selezionati come candidati alla pipeline solo i domini tecnici (coding, math, rights, indici 0–2) la cui probabilità superi la soglia. Il dominio general (indice 3) è escluso per costruzione da questa fase.
- **Stadio 2 — Conferma** (`PIPELINE_PAIR_THRESHOLD = 0.75`, severo): Se lo stadio 1 produce almeno due candidati, viene identificata la coppia con le due probabilità più alte. Questa coppia viene confermata come pipeline

solo se appartiene alle coppie supportate (`_PIPELINE_ORDER`) e se *entrambe* le probabilità superano la soglia più alta. La confidenza della pipeline è definita come $\min(p_a, p_b)$.

- **Fallback mono-dominio:** Se la conferma della pipeline fallisce (una delle due probabilità non supera la soglia, oppure la coppia non è supportata), il `class_id` viene determinato dall'argmax dei quattro `domain_probs`. La confidenza mono-dominio è la probabilità del dominio vincente.

La doppia soglia asimmetrica impedisce che un dominio tecnico “debole” (es. `math=0.52` per presenza di parole matematiche di contorno in una query di puro coding) faccia scattare una pipeline non pertinente: serve una confidenza elevata su *entrambi* i domini per giustificare l’attivazione dell’esecuzione multi-agente.

Nota sui valori di default nel codice sorgente: In `nn_classifier.py` le due costanti sono dichiarate come `DOMAIN_THRESHOLD = config.NEURAL_CLASSIFIER_SETTINGS.get('domain_threshold', 0.35)` e `PIPELINE_PAIR_THRESHOLD = config.NEURAL_CLASSIFIER_SETTINGS.get('threshold_pipeline_pair', 0.60)`. I valori letterali 0.35 e 0.60 che compaiono nel codice sono *fallback di .get()*: verrebbero applicati solo se le chiavi corrispondenti fossero assenti da `config.py`. Poiché `config.py` le definisce sempre esplicitamente (0.50 e 0.75), quei due fallback sono di fatto irraggiungibili in produzione — codice morto difensivo, non un valore alternativo realmente in uso. I valori effettivamente attivi restano 0.50 e 0.75.

Principio di Routing Purity: Il meccanismo appena descritto è anche l’unico punto in cui il sistema decide se e verso quali domini instradare una richiesta. A differenza delle versioni precedenti, nessuna euristica Python a valle (sticky routing testuale, promozione score-based, filtri di lunghezza della query) interviene più a correggere o sovrascrivere l’esito di `_derive_class_id()`. Questo principio, definito *Routing Purity*, è trattato nel dettaglio, con le sue implicazioni su `main.py` e sulla gestione della chat history, nel Capitolo 7.

3.5 Scaricamento e Gestione della Memoria

Immediatamente dopo la classificazione, `main.py` invoca `unload_router()`, che rilascia esplicitamente dalla RAM Python sia l’encoder MiniLM (470 MB) sia i pesi del `MultiTaskMLP`, tramite `del` seguita da `gc.collect()` e `torch.cuda.empty_cache()`. Questo rilascio è necessario per mantenere memoria sufficiente al caricamento dei modelli generativi nell’architettura di sviluppo a 8 GB.

Il costo di questa scelta è un ricaricamento all’inizio del turno successivo (1–3 s), accettabile sul vincolo hardware di sviluppo. In deployment su workstation con RAM abbondante, `unload_router()` può essere disabilitata mantenendo encoder e pesi residenti in memoria tra un turno e l’altro.

Capitolo 4

Pipeline di Addestramento del Classificatore

4.1 Panoramica della Cascata di Preprocessing

L'addestramento del `MultiTaskMLP` segue una cascata di quattro step distinti, ognuno dei quali produce un artefatto intermedio consumato dallo step successivo:

`build_dataset_v2.py` → `precompute_embeddings.py` → `train_nn.py` → `step4_evaluation.py`

Qualsiasi modifica ai dati sorgente (`db_query.py` o `MANUAL_RECORDS` in `build_dataset_v2.py`) richiede la riesecuzione completa della cascata a partire dal primo step.

4.2 Generazione del Dataset (`build_dataset_v2.py`)

Lo script aggrega tre sorgenti distinte in un unico file `dataset_v2.jsonl`:

- **INTENT_SENTENCES** da `db_query.py`: frasi mono-dominio, una per ogni dei quattro domini.
- **BRIDGE_SENTENCES** da `db_query.py`: frasi di confine inter-dominio, etichettate come multi-label e, se pertinenti, come pipeline attiva.
- **MANUAL_RECORDS**: record costruiti tramite tre costruttori tipizzati:
 - `_fu(query, labels, diff, hist)`: record di follow-up genuino (history presente, `is_followup=True`).
 - `_cd(query, labels, diff, hist)`: record di cambio dominio (history presente ma `is_followup=False`).
 - `_r(query, labels, diff)`: record base senza history.

Ogni record del JSONL contiene: `query`, `history` (lista di stringhe), `domain_labels` (dizionario multi-label 0/1), `is_pipeline`, `pipeline_type`, `difficulty`, `is_followup` e `split`.

Etichettatura della difficoltà (`estimate_difficulty()`) — un’euristica ibrida, non puramente strutturale: Il campo `difficulty` non è determinato da un criterio strutturale isolato, né da una costante fissa per dominio, ma dalla combinazione di due segnali distinti. Il livello 3 scatta automaticamente per ogni record di pipeline (`is_pipeline=True`), indipendentemente dal contenuto testuale — questa è la componente strutturale. Per i record mono-dominio, invece, la funzione confronta la query normalizzata in minuscolo con il dizionario lessicale `_HARD_MARKERS`: un insieme di locuzioni indicative di complessità teorica specifiche per ciascun dominio (es. “teorema”, “dimostrazione per induzione” per `math`; “programmazione dinamica”, “crittografia ellittica” per `coding`). Se almeno un marker compare nella query, il livello base del dominio sale di un gradino (general: $1 \rightarrow 2$; domini tecnici: $2 \rightarrow 3$); in assenza di marker resta al valore base. Il sistema è dunque un ibrido strutturale-lessicale, non un’euristica puramente strutturale.

Questo criterio ha sostituito una precedente costante fissa (livello 2 per ogni query tecnica, 1 per general), scartata perché non forniva alla *difficulty head* alcun segnale realmente distinto dall’etichetta di dominio. È stato inoltre preferito a un criterio basato sulla lunghezza della query, che avrebbe penalizzato ingiustamente le numerose query tecniche in formato breve (*short-form*) presenti nel dataset. Va segnalato con onestà che l’euristica lessicale non è stata validata empiricamente su larga scala: la sua efficacia reale richiede verifica tramite smoke test dedicati prima di poterla considerare definitiva, come annotato esplicitamente nel codice sorgente.

Principio di anti-leakage (`split before augmentation`): Lo split stratificato 70/15/15 viene assegnato *prima* della fase di augmentation lessicale. Quest’ultima genera varianti quasi-identiche (un solo sinonimo sostituito) di frasi già presenti nel dataset; eseguire lo split a posteriori consentirebbe a una frase originale e alla sua variante di finire rispettivamente in train e test, gonfiando artificialmente le metriche di validazione.

4.3 Pre-calcolo degli Embedding (`precompute_embeddings.py`)

Lo script carica `dataset_v2.jsonl` e codifica ogni record tramite `paraphrase-multilingual-MiniLM` chiamando `history_utils.build_input_str()` per costruire la stringa di input nel formato identico a quello usato a inferenza. Gli embedding L2-normalizzati (shape $[N, 384]$) e i tensori delle label (`domain`, `difficulty`, `is_followup`) vengono salvati in `classifier/embeddings_v2.pkl` insieme agli indici di split.

Una guardia difensiva verifica che ogni record del JSONL contenga il campo `is_followup`: l’assenza di tale campo provoca un’eccezione esplicita, intercettando regressioni nel generatore del dataset prima che si propaghino silenziosamente all’addestramento.

4.4 Addestramento (`train_nn.py`)

Il training utilizza l’ottimizzatore Adam ($LR = 10^{-3}$, $\text{weight decay} = 10^{-4}$) con lo scheduler `ReduceLROnPlateau` (modalità max, patience 10, fattore 0.5). La loss composita è:

$$\mathcal{L} = 0.70 \cdot \mathcal{L}_{\text{domain}} + 0.30 \cdot \mathcal{L}_{\text{diff}} + 0.15 \cdot \mathcal{L}_{\text{followup}}$$

dove $\mathcal{L}_{\text{domain}}$ è una `BCEWithLogitsLoss` multi-label con `pos_weight` per bilanciare le classi, e $\mathcal{L}_{\text{diff}}$ è una `CrossEntropyLoss`. Il `pos_weight` è calcolato come $\min\left(\frac{N - \text{pos}}{\text{pos}}, \text{MAX_POS_WEIGHT}\right)$, con `MAX_POS_WEIGHT` = 5.0 per entrambi i task.

Il cap è motivato da uno squilibrio di classe marcato: nel dataset generato da `build_dataset_v2.py`, la classe positiva `is_followup=True` rappresenta circa il 14% dei record totali, contro un residuo di circa l’86% di record negativi. Senza un tetto, il `pos_weight` calcolato su questo rapporto spingerebbe i logit della classe minoritaria artificialmente in alto anche su esempi ambigui, causando falsi positivi su `is_followup` sia nelle query brevi prive di storia sia nei cambi di dominio con history presente. Il cap a 5.0 preserva parte del beneficio del riequilibrio (recall sui follow-up corti tipo “perché?”) limitando la distorsione sistemica sul resto della distribuzione.

L’early stopping monitora il F1-macro domain sul validation set con patience 25 epoche. Il checkpoint con il miglior F1 viene salvato in `classifier/nn_weights.pt`.

4.5 Valutazione Qualitativa (`step4_evaluation.py`)

La suite di smoke test comprende 59 query strutturate in sei categorie:

- **A. Mono-domain chiari (20):** verifica della classificazione base su query inequivocabili dei quattro domini.
- **B. Pipeline ibride esplicite (9):** verifica dell’attivazione corretta delle tre pipeline supportate.
- **C. False pipeline (6):** verifica che query ambigue (con keyword di due domini) non attivino erroneamente una pipeline.
- **D. Follow-up ambigui (10):** verifica che `is_followup=True` venga prodotto correttamente in presenza di history.
- **E. Cambio dominio (6):** verifica che query non correlate alla history producano `is_followup=False`.
- **F. Edge case critici (8):** casi noti difficili (keyword fuorvianti, query brevissime, context switch tecnico).

L’accuratezza complessiva della versione attuale è del 91.5% (54/59 test), con accuratezza di routing effettivo del 93.2% (55/59), poiché il flag `is_followup` non influenza il dominio di instradamento. I 4 errori residui appartengono alla sottocategoria “coding history + query corta/generica → general” e sono stati classificati come dataset gap, da risolvere con patch future basate su log di produzione reali.

Capitolo 5

Esecuzione della Pipeline Sequenziale in Ambienti a Memoria Vincolata

5.1 Architettura Draft & Merge: Esecuzione Asincrona

Qualora il classificatore neurale identifichi una richiesta ibrida (`class_id` $\in \{4, 5, 6\}$), l'orchestratore sospende la logica a singola istanza per inizializzare la Pipeline Sequenziale. Questo paradigma si fonda su un'esecuzione sequenziale a due fasi. Il primo LLM invocato (Agente A, determinato dal `class_id`) riceve un prompt direzionale restrittivo (Directional Prompt). Le istruzioni impongono l'inibizione di qualsiasi formula di cortesia o conclusione deduttiva: l'agente è incaricato esclusivamente di generare un elaborato tecnico intermedio (Draft). Questa bozza informativa è strutturata non per la lettura umana, ma per essere elaborata programmaticamente dal nodo successivo. L'intera fase generativa intermedia viene occultata all'utente finale tramite un'interfaccia di caricamento asincrona (Spinner Context), mantenendo pulito l'output a terminale.

5.2 Sincronizzazione Attiva e Gestione dello Scaricamento (RAM)

L'orchestrazione di molteplici LLM su macchine dotate di soli 8 GB di memoria RAM rappresenta la sfida architetturale più complessa del sistema. L'inizializzazione simultanea di due modelli comporterebbe un'immediata saturazione della memoria di sistema (RAM) e il conseguente crash dell'applicativo (Out of Memory).

Per ovviare a questo limite, l'architettura mantiene il meccanismo di deallocazione a due step:

- **Unload Esplicito a Due Step:** Oltre al parametro `keep_alive=0` inviato nell'ultima richiesta all'Agente A, il sistema invoca il metodo `explicit_unload()` in `BaseAI`, che invia una chiamata API dedicata (`ollama.generate` con `prompt=""` e `keep_alive=0`) forzando il sistema di inferenza a scaricare immediatamente i pesi del modello.

- **Gestione del Kernel e Polling:** Viene introdotta una pausa fissa (`ram_unload_wait`, default 3 s) per consentire al kernel di reclamare fisicamente le pagine di memoria `mmap`.

Successivamente, `main.py` avvia il ciclo di Active Polling tramite `psutil`. `psutil.virtual_memory` su sistemi Linux include correttamente cache e buffer recuperabili. Il thread di esecuzione rimane sospeso finché la RAM libera non supera la soglia di sicurezza, entro il limite definito da `ram_sync_timeout`.

5.3 Passaggio di Consegne (Handoff) e Salvaguardia del Contesto

La convergenza dei dati (Merge) si realizza iniettando la bozza generata dall'Agente A all'interno del contesto operativo dell'Agente B. Il prompt di passaggio di consegne (Handoff) include direttive di inibizione della verbosità: all'Agente B è fatto divieto di reiterare spiegazioni tecniche già fornite dal predecessore, ottimizzando la sintesi finale.

Contestualmente, è attivo un meccanismo di salvaguardia contro la saturazione della finestra di contesto (Context Window). Se l'output dell'Agente A risulta eccessivamente prolisso, la classe `BaseAI` interviene tramite il metodo `_truncate_context()`, che recide l'elaborato in eccesso basandosi sul limite dinamico `pipeline_max_context_chars` definito in `config.py`. Questa astrazione garantisce la stabilità attentiva dell'Agente B anche su hardware con risorse limitate.

Capitolo 6

Chat History e Persistenza Conversazionale

6.1 Architettura della Memoria di Sessione

CYA N implementa un sistema di gestione della memoria a breve termine per supportare interazioni multi-turno. La chat history è strutturata come una lista globale di sessione gestita dal modulo `main.py`, composta da una sequenza di dizionari nel formato nativo richiesto dai motori di inferenza (`{"role": str, "content": str}`).

Per scelta architetturale legata alla privacy e alla velocità, la cronologia non viene persistita su disco: risiede esclusivamente nella RAM e viene azzerata alla chiusura del processo o tramite comando esplicito.

6.2 Meccanismo di Sliding Window

Per garantire la stabilità del sistema su hardware con soli 8 GB di RAM e prevenire l'overflow della Context Window (fissata a 4096 token), la cronologia è regolata da una finestra scorrevole (Sliding Window). Il parametro `max_history_turns` in `config.SYSTEM_SETTINGS` (default: 5) definisce la profondità della memoria. Uno “scambio” completo comprende il messaggio dell'utente e la risposta dell'assistente; pertanto, un'impostazione di 5 turni comporta il mantenimento di un massimo di 10 messaggi. Quando la soglia viene superata, il sistema provvede alla rimozione automatica dei messaggi più obsoleti.

Dalla versione 7.5.0, il limite sul *numero* di messaggi è affiancato da un cap sulla *lunghezza* del singolo messaggio: la funzione `_truncate_for_history()` in `main.py` tronca ogni messaggio (utente o assistente) a `config.SYSTEM_SETTINGS['history_message_max_characters']` (default 1200) prima di inserirlo in `chat_history`, tramite `_update_history()`. In assenza di questo limite, una risposta particolarmente estesa (un blocco di codice complesso, una spiegazione giuridica articolata), combinata con una sliding window fino a 10 messaggi, poteva saturare silenziosamente la finestra di contesto (`ctx_size = 4096` token): Ollama tronca tipicamente dall'inizio del contesto, con il rischio concreto di perdere lo stesso system prompt senza alcun errore visibile. Il troncamento agisce esclusivamente su ciò che viene ricordato nei turni successivi: il

testo integrale della risposta resta comunque visibile all'utente durante lo streaming in `ai_engine.py::generate()`.

Vale la pena distinguere la profondità usata dal classificatore da quella usata dai modelli generativi: `history_utils.HISTORY_MAX_TURNS = 2` determina quante query precedenti l'encoder MiniLM considera nella stringa di input per la classificazione, mentre `max_history_turns` governa l'intera sliding window passata ai modelli generativi. I due parametri sono indipendenti e regolano aspetti distinti del sistema.

6.3 Integrazione e Iniezione del Contesto

La history viene iniettata dinamicamente in ogni chiamata ai modelli. Questa procedura coinvolge i metodi `resolve()`, `resolve_pipeline_a()` e `resolve_pipeline_b()`.

- **Fusione Few-Shot:** Al fine di evitare conflitti di ruolo tra gli esempi predefiniti e i turni reali della conversazione, il metodo `_merge_few_shot()` in `BaseAI` fonde gli esempi few-shot direttamente nel System Prompt. Questo garantisce che gli esempi virtuosi fungano da istruzione di base senza alterare la cronologia temporale dei messaggi.
- **Esclusione del Critic Pass:** Per design deliberato, il metodo `execute_critic_pass()` non riceve la cronologia. La revisione critica deve restare focalizzata esclusivamente sulla coerenza tra la bozza corrente e la richiesta originale dell'utente, evitando che il contesto pregresso possa distorcere o “ammorbidire” la severità della validazione.

6.4 Integrità dello Stato e Controllo Utente

Il sistema protegge la qualità della memoria conversazionale tramite un aggiornamento condizionale: la funzione `_update_history()` viene invocata esclusivamente se la risposta prodotta non è classificata come errore di sistema (`_is_error`). Questo impedisce che messaggi diagnostici (es. “OOM RAM insufficiente”) vengano memorizzati, preservando la coerenza logica dei turni successivi.

L'utente dispone inoltre del comando speciale `/reset` (o `/clear`) a runtime. L'esecuzione di tale comando svuota istantaneamente la chat history e azzerla la variabile `last_active_domain`, permettendo di iniziare una nuova sessione tematica senza dover riavviare l'intera infrastruttura.

Capitolo 7

Gestione del Cambio di Dominio e Isolamento del Contesto

7.1 Principio di Routing Puro

A partire dalla versione 7.4.0, il routing di CYA N è governato da un principio di purezza: il `class_id` prodotto da `nn_classifier.predict()` è l'unica variabile che determina il percorso esecutivo. Non esiste alcuna logica Python successiva che possa alterare, correggere o sovrascrivere questa classificazione. Le variabili `domain_scores`, `difficulty` e `is_followup` transitano in `main.py` unicamente come informazioni diagnostiche visualizzate nel log di sessione.

Questo approccio semplifica radicalmente l'architettura rispetto alle versioni precedenti, in cui euristiche multi-step (Domain Retention, promozione score-based, filtri di complessità) intervenivano *post-classificazione* per alterare il dominio scelto. La responsabilità di gestire follow-up, query brevi e cambi di argomento è stata interiorizzata nel classificatore neurale, addestrato su record espliciti per ognuna di queste sotto-categorie.

7.2 Isolamento della History su Domain Switch

L'unica responsabilità di `main.py` legata al dominio è la gestione della variabile `domain_switched`. Quando il dominio del turno corrente differisce dall'ultimo dominio attivo (`last_active_domain`), viene impostata la flag `domain_switched = True`. Questa condizione non altera il dominio di instradamento, ma determina quale slice della chat history viene passata all'agente: se `domain_switched` è vera, l'agente riceve una history vuota (`effective_history = []`), isolando il contesto del nuovo dominio da risposte di un dominio differente.

Questo meccanismo previene la “contaminazione semantica” dei context switch: un agente di coding non dovrebbe ricevere come contesto le risposte giuridiche del turno precedente, e viceversa.

Fino alla versione 7.3.x l'isolamento era implementato solo nel ramo di esecuzione mono-dominio; una richiesta che attivava una pipeline ibrida subito dopo un turno mono-dominio di un dominio diverso poteva quindi ricevere una history non isolata nella Fase 1 (Agente A). Dalla versione 7.5.0 lo stesso confronto

(`last_active_domain` rispetto al primo dominio della coppia, `domain_a`) è applicato anche al ramo `pipeline`, e la medesima `effective_history` isolata viene passata sia a `resolve_pipeline_a()` sia a `resolve_pipeline_b()`, eliminando l'asimmetria tra i due rami.

7.3 Aggiornamento dello Stato di Sessione

La variabile `last_active_domain` viene aggiornata a ogni risposta conclusa con successo (`not _is_error(result)`), includendo il dominio `general`. Per le `pipeline`, viene aggiornata con `domain_b` (il dominio dell'Agente B, che produce l'output finale). L'utente può azzerare esplicitamente lo stato con il comando `/reset`.

Capitolo 8

Critic Pass, Analisi Strutturata e Sanificazione dell'Output

8.1 Revisione Critica: Validazione Antagonistica (Critic Pass)

Nei sistemi ad agenti sequenziali, sussiste il rischio architetturale noto come “Fiducia Cieca” (Blind Trust), fenomeno per cui il nodo finale assimila e riproduce acriticamente i dati forniti dal nodo precedente, propagando eventuali allucinazioni semantiche.

Per interrompere questa catena, CYA N implementa una fase di convalida autonoma denominata Critic Pass. Sfruttando la permanenza in RAM dell'Agente B, l'orchestratore gli impone di rianalizzare il proprio output finale prima della renderizzazione a schermo. Il prompt di validazione (`PIPELINE_PROMPTS['critic']`) inietta dinamicamente la stringa `{original_query}` formulata dall'utente. Questa iniezione funge da ancoraggio semantico: l'LLM è costretto a confrontare la sua sintesi con l'intento originale, verificando la correttezza fattuale dell'integrazione e colmando eventuali deviazioni.

Le direttive impongono inoltre la correzione silente: se non vengono rilevate anomalie, l'agente restituisce il testo invariato astenendosi dal produrre metadati valutativi (es. “Il testo risulta corretto”).

8.2 Parsing Strutturato e Gestione dei Tag di Ragionamento

L'adozione di modelli analitici avanzati comporta l'emissione di token di ragionamento latente racchiusi all'interno di tag strutturati (es. `<think></think>`). La logica di parsing a livello di streaming in `ai_engine.py` è astratta: i tag non sono codificati rigidamente, ma derivati dalle variabili `think_open_tag` e `think_close_tag` in `config.SYSTEM_SETTINGS`.

Sanificazione Code-Block Aware: La routine di sanificazione opera conversioni sui simboli LaTeX e sui logogrammi asiatici (CJK). La funzione `clean_response()`

è pienamente consapevole della struttura Markdown: il testo viene diviso tramite una regex sui delimitatori backtick. Le sostituzioni LaTeX $\rightarrow$ Unicode e il filtro CJK vengono applicati esclusivamente alle sezioni di testo discorsivo, preservando intatti i blocchi di codice. Questo previene la corruzione di snippet tecnici contenenti simboli matematici o caratteri orientali. Il filtro CJK è controllato dal flag `cjk_filter_enabled` in `SYSTEM_SETTINGS` (default True). Quando il filtro rimuove caratteri, l'evento viene loggato (conteggio e anteprima dei caratteri rimossi) per favorirne il debug: il filtro resta intenzionalmente globale su tutti i domini, incluso `general` — che tratta anche argomenti dove sequenze CJK legittime possono comparire, ad esempio riferimenti al galateo giapponese — piuttosto che condizionato per dominio, scelta che avrebbe richiesto propagare la categoria in ogni punto di chiamata di `clean_response()`.

8.3 Intercettazione delle Eccezioni a Basso Livello

L'architettura rafforza la gestione degli stati di anomalia (Error Handling). Se durante le operazioni di pre-flight la funzione `check_resources()` rileva indisponibilità critica di RAM non risolvibile, il motore `ai_engine.py` solleva l'eccezione tipizzata `ResourceExhaustedError`.

L'intercettazione di questo errore nel blocco try-except di `main.py` blocca istantaneamente il task corrente, impedendo all'orchestrazione di passare un messaggio di errore generico all'Agente B come se fosse una bozza valida da elaborare, garantendo così una chiusura controllata ed elegante (Graceful Degradation).

Capitolo 9

Topologia della Flotta LLM e Parametri di Inferenza

9.1 Provisioning e Gerarchia dei Modelli

L’infrastruttura di CYA N non converge su un modello generalista monolitico, bensì su una topologia a sciame (Swarm Topology) composta da modelli locali quantizzati, definiti dinamicamente nel modulo `config.py`. La distribuzione del carico cognitivo è organizzata secondo una gerarchia di competenza e requisiti hardware. Il classificatore neurale, non essendo un modello generativo Ollama, non fa parte della flotta LLM: viene caricato e scaricato autonomamente da `nn_classifier.py` prima che qualsiasi agente generativo entri in scena.

- **Dominio Informatica (Coding):** Il nodo esecutivo primario è `qwen3.5:9b`. Per garantire la resilienza su macchine a basse prestazioni, il sistema prevede un fallback hardware: se la RAM disponibile rilevata scende sotto la soglia `medium` (5.5 GB), l’istanziatura viene deviata sul modello `qwen2.5-coder:1.5b`, ottimizzato per spazi di memoria estremamente ristretti.
- **Dominio Logico-Matematico (Math):** La risoluzione di problematiche inerenti l’analisi complessa e la statistica è affidata in via primaria a `deepseek-r1:7b`, dietro la soglia dedicata `math_opt` (4.5 GB) — più alta della soglia `small` ma inferiore a `medium`, per riflettere un requisito di RAM intermedio tra i modelli da 1.5B e quelli da 9B/20B. Se la RAM disponibile scende sotto tale soglia, il sistema declassa al modello di fallback `deepseek-r1:1.5b` (soglia `small`, 1 GB), scelto per preservare la stessa famiglia di modello e quindi lo stile di ragionamento a catena, a costo di una minore accuratezza sui problemi più complessi.
- **Domini Normativo e Generalista (Rights e General):** Per coprire la vasta mole di giurisprudenza e cultura generale, l’architettura fa affidamento su `gpt-oss:20b`. Considerando l’imponente richiesta di memoria, sussiste un controllo di sbarramento a 12 GB di RAM (`large`): al di sotto di tale limite, il traffico viene reindirizzato in sicurezza verso il più snello `gemma3:4b`.

Nota sulla Configurazione Hardware-Aware: Sebbene l’hardware di sviluppo di riferimento disponga di soli 8 GB di RAM, `config.py` dichiara già la topologia a sciame completa descritta sopra — un modello primario dedicato per dominio, non una configurazione di test statica e ridotta a un unico modello leggero condiviso. Il vincolo hardware viene gestito *a runtime* tramite `RAM_THRESHOLDS` e il meccanismo di downgrade preventivo di `check_resources()` (Capitolo 1): su una macchina a 8 GB è atteso che i domini con soglia `large` (12 GB, Rights/General) e in parte `medium` (5.5 GB, Coding) operino prevalentemente sul rispettivo modello di fallback, mentre il dominio Math (soglia `math_opt`, 4.5 GB) dispone di un margine relativamente più ampio. La piena operatività sui soli modelli primari è raggiunta su workstation con RAM sufficiente a soddisfare simultaneamente le soglie più esigenti (idealmente almeno 16 GB dedicati).

9.2 Calibrazione delle Temperature (Determinismo vs Entropia)

Il parametro di Temperatura (T) modula la distribuzione di probabilità dei token in fase di inferenza, determinando il bilanciamento tra rigore analitico e creatività discorsiva. L’architettura assegna indici termici differenziati per ciascun dominio operativo:

- **T=0.2 (Math):** Impostazione prossimo-deterministica. Sfruttata per inibire l’entropia semantica, risulta indispensabile per l’enunciazione di formule, dimostrazioni e teoremi, ambiti in cui l’esattezza strutturale non ammette variazioni stocastiche.
- **T=0.4 (Rights):** Entropia vincolata. Fornisce all’agente sufficiente elasticità lessicale per articolare concetti giuridici in prosa, mantenendo un vincolo probabilistico stringente che impedisce la sintesi allucinatoria di testi di legge inesistenti.
- **T=0.5 (Coding):** Modulazione ibrida. Consente l’esplorazione di design pattern e soluzioni algoritmiche laterali, pur vincolando la generazione sintattica per garantire la validità del codice emesso.
- **T=0.7 (General):** Massima varianza autorizzata. Applicato al dominio generalista per sbloccare la naturalezza colloquiale e l’eterogeneità del vocabolario.

Il classificatore neurale non è soggetto a temperatura: l’inferenza del `MultiTaskMLP` è deterministica per natura, producendo classificazioni identiche a parità di input indipendentemente dalla sessione.

9.3 Sistema di Prompt Tiering

Complementare alla scelta del modello e della temperatura, `config.py` associa a ciascun dominio in `MODELS_CONFIG` un campo statico `prompt_tier` (`compact op-`

pure `extended`), che governa esclusivamente il contenuto del system prompt costruito da `prompts_templates.py::get_prompts()` — nessun impatto su routing, classificatore neurale o gestione della RAM.

`BaseAI.__init__` (`ai_engine.py`) legge `self.prompt_tier` una sola volta all'istanziamento dell'agente, con `'compact'` come default difensivo per un dominio privo del campo. Il valore resta invariato per l'intera vita dell'istanza, *indipendentemente* dallo stato `is_using_fallback`: un downgrade runtime verso il modello di riserva non declassa mai il tier a `compact`. Questa scelta riflette un'asimmetria di rischio tra le due direzioni d'errore: un prompt `extended` somministrato a un modello di fallback più piccolo comporta, nel caso peggiore, l'ignorare parte delle istruzioni di arricchimento; vincolare invece un modello primario capace a un prompt `compact` ne sacrificerebbe la qualità potenziale senza necessità.

I `SYSTEM_PROMPTS` di base in `prompts_templates.py` sono scritti in versione compatta (nessuna teoria introduttiva non richiesta, struttura minima), ottimizzata per modelli di fascia contenuta. L'arricchimento per modelli più capaci vive esclusivamente nel dizionario `TIER_INJECTIONS`: `get_prompts(category, tier)` concatena il blocco corrispondente in coda al prompt base *solo se* `tier == 'extended'`, mai in sostituzione. Ogni blocco di iniezione termina con una nota di sicurezza (`_TIER_PIPELINE_SAFETY_NOTE`) che ne vincola l'effetto al solo contenuto tecnico quando il prompt è usato in modalità pipeline, dove `PIPELINE_PROMPTS['directional']` vieta esplicitamente introduzioni, saluti o conclusioni discorsive.

L'assegnazione corrente per dominio è: `coding = extended, rights = extended, general = extended, math = compact`. Il caso `math` è l'unico non allineato al resto: `deepseek-r1:7b` esegue nativamente un ragionamento a catena (chain-of-thought) racchiuso nei tag `<think>...</think>`, rimossi da `clean_response()` (`helper.py`) prima della visualizzazione. Un'iniezione `extended` che richiedesse un'ulteriore premessa teorica risulterebbe ridondante rispetto al ragionamento già interno al modello, per cui il tier `compact` resta la scelta corretta per questo dominio.

Tutti i quattro metodi pubblici (`resolve`, `resolve_pipeline_a`, `resolve_pipeline_b`, `execute_critic_pass`) di ciascuna delle tre classi agente (`CodeLlamaAI`, `DeepSeekAI`, `GptOssAI`) passano `self.prompt_tier` come secondo argomento a `get_prompts()`, per un totale di dodici call-site coerenti.

Capitolo 10

Istruzioni Operative: Messa in Produzione

10.1 Inizializzazione del Motore di Inferenza (Ollama)

Il prerequisito fondamentale per il dispiegamento dell'infrastruttura generativa di CYA N è l'installazione e la configurazione del motore di inferenza locale. Nel rispetto del vincolo di totale indipendenza dalle reti esterne, l'architettura adotta Ollama come demone locale per l'esecuzione dei modelli generativi. Il classificatore neurale (`nn_classifier.py`) è invece indipendente da Ollama e viene eseguito direttamente tramite PyTorch.

10.2 Provisioning della Flotta: Scaricamento dei Modelli

Una volta instaurata la comunicazione con il demone locale, è necessario allocare sull'hardware i pesi neurali dei modelli generativi previsti dall'architettura V7.5.0. L'allocazione avviene da terminale tramite i seguenti comandi:

- **Dominio Informatica (Coding):** `ollama pull qwen3.5:9b` e `ollama pull qwen2.5-coder:1.5b`
- **Dominio Logico-Matematico (Math):** `ollama pull deepseek-r1:7b` e `ollama pull deepseek-r1:1.5b`
- **Domini Rights e General:** `ollama pull gpt-oss:20b` e `ollama pull gemma3:4b`

10.3 Addestramento del Classificatore Neurale

Prima del primo avvio dell'applicazione è necessario disporre del checkpoint `classifier/nn_weights`. La procedura di addestramento richiede l'esecuzione sequenziale dei seguenti comandi dalla root del progetto:

```
python code/build_dataset_v2.py
python code/precompute_embeddings.py
python code/train_nn.py
python code/step4_evaluation.py
```

L'ultimo step è opzionale ma consigliato: produce il report di accuratezza e il file `wrong_query_TESTING.md` con i test falliti.

10.4 Configurazione dell'Ambiente e Avvio

Il software richiede un ambiente Python 3.8 o superiore. Le dipendenze di sistema includono:

```
pip install psutil ollama sentence-transformers torch scikit-learn
```

È fondamentale evidenziare il ruolo di ciascuna libreria critica:

- **psutil**: modulo di telemetria hardware del sistema. In sua assenza, CYA N perderebbe la capacità di interrogare la memoria RAM a runtime, inibendo i declassamenti preventivi, le procedure di Active Polling e lo scaricamento esplicito (Explicit Unload), esponendo la macchina a un inevitabile blocco per saturazione della memoria.
- **sentence-transformers** e **torch**: librerie necessarie sia per l'addestramento del `MultiTaskMLP` sia per l'inferenza in produzione. Il modello `paraphrase-multilingual-MiniLM` viene scaricato automaticamente da HuggingFace al primo utilizzo.
- **scikit-learn**: utilizzata in `train_nn.py` per il calcolo dell'F1-score durante il training e l'early stopping.

Completata l'installazione e l'addestramento del classificatore, l'infrastruttura viene avviata dal nodo principale:

```
python code/main.py
```

Capitolo 11

Glossario Architetture

Arco di Fallback Hardware Il meccanismo di mitigazione dei guasti che inter-cetta a runtime una riduzione critica della RAM disponibile e reindirizza le operazioni verso un modello generativo di taglia inferiore. Il fallback testuale a parole chiave, presente nelle versioni precedenti, è stato rimosso: se il classificatore neurale non è disponibile, il turno viene scartato con un errore esplicito.

Backbone Condiviso Il sotto-reti iniziale del `MultiTaskMLP` (strati $384 \rightarrow 256 \rightarrow 128$) i cui pesi vengono aggiornati da tutti e tre i task simultaneamente. Apprende una rappresentazione latente condivisa da cui le tre teste specializzate derivano le proprie predizioni.

`build_input_str()` Funzione definita in `history_utils.py` e condivisa tra la fase di training (`precompute_embeddings.py`) e quella di inferenza (`nn_classifier.py`). Costruisce la stringa di input passata all'encoder MiniLM nel formato `[HISTORY] q1 | q2 [QUERY] query` (o solo la query, se la history è vuota).

Chat History (Sliding Window) Il buffer di sessione conversazionale gestito da `main.py`. Mantiene gli ultimi `max_history_turns` scambi user/assistant per supportare il dialogo multi-turno. Non è persistita su disco. Viene isolata (passata vuota all'agente corrente) in caso di domain switch.

`class_id` L'identificatore intero prodotto da `nn_classifier.predict()` che determina univocamente il percorso di esecuzione. Valori 0–3 corrispondono ai quattro domini mono (coding, math, rights, general); valori 4–6 identificano le pipeline (math→coding, rights→coding, rights→math). È l'unica fonte di verità per il routing: nessun modulo lo altera a posteriori.

Critic Pass Modulo di autovalutazione antagonista che forza l'Agente B a validare il proprio output confrontandolo con la richiesta originale per mitigare le allucinazioni. Non riceve la chat history per design deliberato.

`domain_switched` Flag booleana in `main.py` che segnala un cambio di dominio rispetto all'ultimo turno. Non altera mai il `class_id` o il dominio di instradamento; determina esclusivamente se la chat history viene passata vuota all'agente corrente (isolamento del contesto).

Explicit Unload Metodo `explicit_unload()` in `BaseAI` che forza il rilascio immediato del memory mapping dei tensori del modello generativo dopo il completamento dell'Agente A, riducendo la latenza del polling RAM. Distinto da `unload_router()`, che riguarda invece il classificatore neurale.

history_utils.py Modulo condiviso tra pipeline di training e inferenza. Unica fonte di verità per `build_input_str()` e `HISTORY_MAX_TURNS`. Garantisce che l'embedding calcolato a training sia identico a quello calcolato in produzione.

MultiTaskMLP Il classificatore neurale leggero (145K parametri) che costituisce il cuore del routing di CYA N V7.5.0. Composto da un backbone condiviso e tre teste specializzate (domain, difficulty, is_followup). Opera su embedding L2-normalizzati a 384 dimensioni prodotti dall'encoder MiniLM frozen.

PIPELINE_CLASSES Dizionario pubblico in `nn_classifier.py` che mappa i `class_id` pipeline (4, 5, 6) alle coppie di domini (`domain_a`, `domain_b`). Consumato da `main.py` per determinare quali agenti attivare e in quale ordine.

pos_weight Coefficiente di bilanciamento delle classi nella `BCEWithLogitsLoss`. Calcolato come rapporto negativi/positivi per ogni task e cappato a `MAX_POS_WEIGHT = 5.0` per prevenire la distorsione sistematica dei logit che causerebbe falsi positivi su `is_followup`.

Principio di Anti-Leakage Regola architetturale che impone l'assegnazione dello split 70/15/15 *prima* della fase di augmentation lessicale in `build_dataset_v2.py`, per evitare che varianti quasi-identiche di una stessa frase contaminino il set di test.

Prompt Tiering Meccanismo statico (`config.py`, `ai_engine.py`, `prompts_templates.py`) che assegna a ciascun dominio un livello di arricchimento del system prompt (`prompt_tier`: `compact` o `extended`), letto una sola volta da `BaseAI.__init__` e indipendente dallo stato di fallback runtime. Il tier `extended` concatena in coda al prompt base un blocco `TIER_INJECTIONS[category]` additivo, mai sostitutivo.

Stateful (Con stato) Paradigma in cui le istanze dei modelli generativi vengono preservate “calde” in RAM per ottimizzare la latenza delle interazioni successive.

unload_router() Funzione pubblica di `nn_classifier.py` che rilascia esplicitamente encoder MiniLM e pesi MLP dalla RAM Python tramite `del + gc.collect() + torch.cuda.empty_cache()`. Va chiamata subito dopo la classificazione, prima che Ollama carichi il modello generativo dell'agente.